
Algoritmos de Ordenação: Eficiência e Complexidade

Relatório Técnico Acadêmico | ADS 2026
Selection / Bubble / Insertion / Merge / Quick Sort

30 de março de 2026

Este documento apresenta uma síntese técnica da Unidade 2, Seção 3 da disciplina Linguagem de Programação, baseando-se nos estudos de Vanessa Cadan Scheffer.

1 Objetivos

Este relatório tem como objetivo analisar os principais métodos de ordenação de dados e sua eficiência computacional. Busca-se compreender a diferença entre algoritmos iterativos simples e algoritmos avançados baseados na estratégia de "Dividir para Conquistar", avaliando o impacto da complexidade algorítmica (O) no desempenho de sistemas de software.

2 Desenvolvimento Teórico

De acordo com SCHEFFER (2020), a ordenação é o processo de arranjar elementos em uma sequência específica (crescente ou decrescente) para otimizar buscas e processamentos. A escolha do algoritmo ideal depende diretamente da volumetria dos dados e das restrições de memória.

- **Métodos Simples:** Incluem o *Bubble Sort*, *Selection Sort* e *Insertion Sort*. Possuem complexidade $O(N^2)$, sendo eficientes apenas para pequenos conjuntos de dados devido ao alto custo de comparações e trocas.
- **Métodos Avançados:** O *Merge Sort* e o *Quick Sort* utilizam a recursividade e a partição de problemas. Com complexidade média de $O(N \log N)$, são fundamentais para aplicações de alta performance.

Conceito-Chave: A Complexidade de Algoritmo não mede o tempo em segundos, mas o crescimento do esforço computacional em relação ao aumento do volume de dados (Notação Big-O).

3 Aplicação Prática/Código

A validação prática no Google Colaboratory permitiu comparar o tempo de execução entre os métodos. Enquanto o *Bubble Sort* apresenta um crescimento exponencial de tempo, o *Quick Sort* mantém a estabilidade mesmo com listas extensas.

```
# Exemplo de Quick Sort (Divisão e Conquista)
def quick_sort(lista):
    if len(lista) <= 1:
        return lista

    pivo = lista[len(lista) // 2]
    esquerda = [x for x in lista if x < pivo]
    meio = [x for x in lista if x == pivo]
    direita = [x for x in lista if x > pivo]
    # Retorno com acentuação corrigida via literate
    return quick_sort(esquerda) + meio + quick_sort(direita)

print("Ordenação concluída com eficiência!")
```

Como evidenciado nos testes, a escolha de algoritmos estáveis, como o *Merge Sort*, é preferível quando a ordem relativa de elementos iguais deve ser preservada, conforme discutido por SCHEFFER (2020).

Conclusão

A exploração da Unidade 2.3 revelou que o domínio da complexidade algorítmica é uma competência crítica para o profissional de ADS. A transição de métodos iterativos para métodos de "dividir para conquistar" não é apenas uma escolha técnica, mas uma necessidade estratégica para garantir a escalabilidade e a redução de custos de processamento em infraestruturas modernas.

Referências

1. SCHEFFER, Vanessa Cadan. **Linguagem de Programação**. Unidade 2, Seção 3. Livro Digital. Londrina: Editora e Distribuidora Educacional S.A., 2020.
2. GUIMARÃES, Murilo. **Exercícios Práticos e Simulações**. Google Colaboratory, 2026. Disponível em: https://colab.research.google.com/drive/1M7mWehWw-Dsc8xcQ4PEp_KMMYvW4nhJa?usp=sharing.